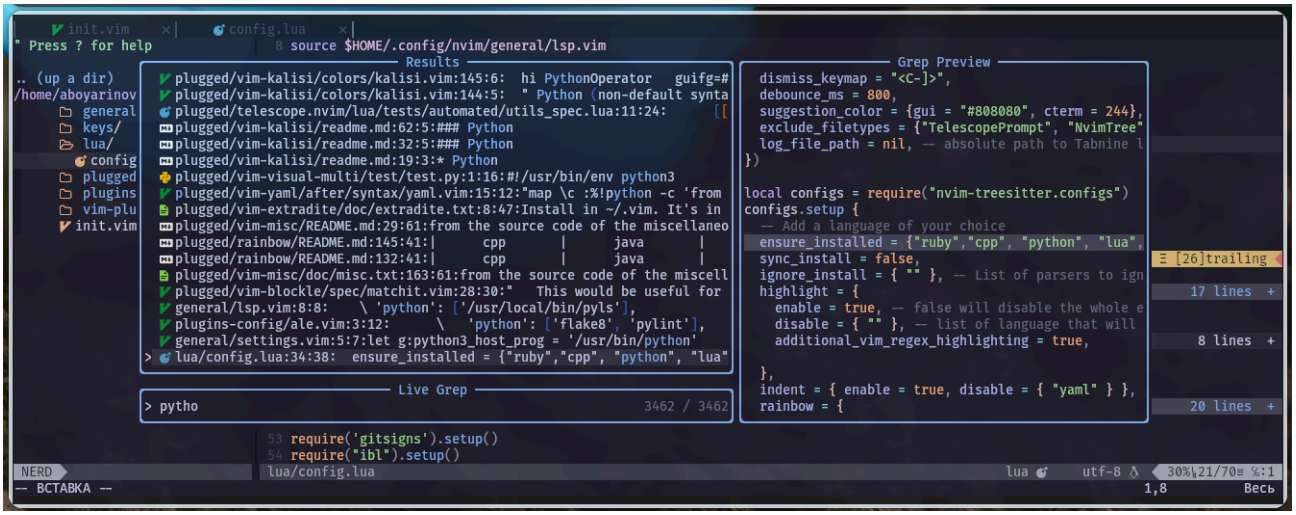
NeoVim для Ruby разработчика

habr.com/ru/articles/797577

March 2, 2024



Данная конфигурация собиралась очень долго. на данный момент работаю именно на ней. Является больше напоминанием для себя, и шпаргалкой для тех кому интересно. Это субъективная точка зрения, и ни кому ее не навязываю. Если захотите использовать, буду рад тем что помог.

Резюме статьи:

- данная сборка используется для разработки на Ruby on Rails, так же есть поддержка сопутствующих языков
- сборка конфигурации NeoVim и его настройка проводится в двух стандартах, по причине выхода новых плагинов и специфики их настройки. есть файлы конфигурации vim а так же lua
- для помощи в написании кода используется в том числе и tabnine
- treesitter используется для расширения поддержки различных языков
- использование некоторых старых плагинов обусловлено спецификой работы, используется slim, а на lua пока нет достойных вариантов, а к некоторым просто привык
- в качестве глобальной системы поиска по всему используется Telescope (поиск по наименованию файлов, содержимому, среди буферов и т.п.)
- По ходу настроек буду писать пути решения проблем для разных систем, систем семейства Debian/Ubuntu, Fedora, Arch, MacOs

Если вас заинтересовал функционал, добро пожаловать в наше маленькое путешествие.

1. Установка NeoVim

Документация по установке NeoVim для различных систем.

В LinuxMint через стандартный менеджер пакетов устанавливается старая версия, 6.5. по этому следущий способ лучше подходит для свежей установки. Нужна версия выше 0.8, так как часть плагинов не будут работать на версиях ниже.

Способ установки последней версии NeoVim из гит репозитория, сборок для скачивания. должно работать на всех системах Lunux,:

```
curl -LO https://github.com/neovim/neovim/releases/latest/download/nvim-
linux64.tar.gz
sudo rm -rf /opt/nvim
sudo tar -C /opt -xzf nvim-linux64.tar.gzОбъяснить c
```

для систем MacOsX:

```
curl -LO https://github.com/neovim/neovim/releases/download/latest/nvim-
macos.tar.gz
tar xzf nvim-macos.tar.gz
./nvim-macos/bin/nvimОбъяснить c
```

после выполнения кода выше необходимо прописать путь к исполняемому файлу (в примере из документации используется bash)

добавить следующую строку в файле **~/.bashrc**:

```
export PATH="$PATH:/opt/nvim-linux64/bin"Объяснить c
```

Установка NeoVim используя пакетный менеджер системы:

Установка для Debian/Ubuntu: `sudo apt-get install neovim`

Установка для Fedora: `sudo dnf install -y neovim`

Установка для Arch: `sudo pacman -S neovim`

Установка для MacOs: `brew install neovim`

2. Подготовка рабочего окружения

Установка для Debian/Ubuntu:

```
sudo apt-get install curl git
sudo apt-get install python-is-python3
sudo apt-get install pip
pip install neovim
Объяснить с
```

```
LAZYGIT_VERSION=$(curl -s
"https://api.github.com/repos/jesseduffield/lazygit/releases/latest" | grep -Po
'"tag_name": "v\K[^"]*')
curl -Lo lazygit.tar.gz
"https://github.com/jesseduffield/lazygit/releases/latest/download/lazygit_${LAZYGI
T_VERSION}_Linux_x86_64.tar.gz"
tar xf lazygit.tar.gz lazygit
sudo install lazygit /usr/local/bin
Объяснить с
```

```
sudo apt-get install ripgrepОбъяснить с
```

Установка для Fedora:

```
sudo dnf install -y python3-neovim
sudo dnf copr enable atim/lazygit -y
sudo dnf install lazygit
sudo dnf install gcchttps://tracker.yandex.ru/agile/board/6
sudo dnf install ripgrep Объяснить с
```

```
sudo dnf install gccОбъяснить с
```

Установка для Arch:

```
sudo pacman -S python-pynvim
sudo pacman -S lazygit
```

```
sudo pacman -S ripgrepОбъяснить с
```

Установка для MacOs:

```
brew install lazygitОбъяснить с
```

у меня при запуске nvim возникли трудности с deoplete, ругался на отсутствие pynvim, решение простое:

```
wget -qO- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.3/install.sh | bash
nvm install 20
npm install -g tree-sitter tree-sitter-cli
Объяснить с
```

Так же для корректного отображения предлагаю использовать iTerm2

Установка Node Version Manager для всех типов систем

```
wget -qO- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.3/install.sh | bash
nvm install 20
npm install -g tree-sitter tree-sitter-cliОбъяснить с
```

после всего описанного выше, запускаем из терминала `nvim` для проверки запуска самого редактора

3. Установка шрифтов для правильного отображения символов и иконок NerdFonts

Шрифты NerdFonts Остановил свой выбор на FiraCode

```
mkdir -p ~/.local/share/fonts
mkdir ~/tmp
wget https://github.com/ryanoasis/nerd-fonts/releases/download/v3.1.1/FiraCode.zip
unzip FiraCode.zip -d ~/tmp
rm FiraCode.zip
mv ~/tmp/FiraCode* ~/.local/share/fonts
rm -rf ~/tmpОбъяснить с
```

Далее для Osx мы выделяем шрифты в finder, открываем в Диспетчере шрифтов и нажимаем установить. Шрифты установятся в пользовательский каталог автоматически

```
rm FiraCode.zip
mv ~/tmp/FiraCode* ~/.local/share/fonts # для Linux
rm -rf ~/tmpОбъяснить с
```

после необходимо закрыть терминал, открыть его заново, поменять шрифт и снова переоткрыть его.

4. Установка конфигурационных файлов

для начала очистим конфигурацию neovim на всякий случай, чтоб избежать трудностей в дальнейшем. Если до этого у вас уже были настройки, мы их сохраним с расширением bak для возможности вернуть все обратно

```
mv ~/.config/nvim ~/.config/nvim.bak
mv ~/.local/share/nvim ~/.local/share/nvim.bak
mv ~/.local/state/nvim ~/.local/state/nvim.bak
mv ~/.cache/nvim ~/.cache/nvim.bakОбъяснить с
```

Скачать архив с конфигурацией для NeoVim

```
wget https://github.com/apilot/dotfiles/releases/download/NeoVim/nvim-
latest.zipОбъяснить с
```

разархивировать скаченный файл сразу в директорию с конфигурацией

```
unzip ./nvim-latest.zip -d ~/.config/ && rm nvim-latest.zipОбъяснить с
```

5. Установка системы пакетов Plug

```
sh -c 'curl -fLo "${XDG_DATA_HOME:-$HOME/.local/share}"/nvim/site/autoload/plug.vim
--create-dirs \
       https://raw.githubusercontent.com/junegunn/vim-
plug/master/plug.vim'Объяснить с
```

6. Установка и обновление пакетов

после всех процедур запускаем в терминале nvim, видим не очень красивые надписи и ошибки. Нажимаем ESC. Далее входим в режим ввода команд, нажимая `:`
и вводим команду `PlugInstall` и ждем завершения установки всех плагинов и того что необходимо для них.
По завершении установки входим в режим ввода команд ':' вводим команду `qa` (что означает выйти из всех открытых окон/вкладок)

Если в Osx возникли ошибки связанные с deoplete то в файле конфигурации нужно поменять путь к python в файле ~/.config/nvim/general/settings изменить путь с `let g:python3_host_prog = '/usr/bin/python'`
на
`let g:python3_host_prog = '~/.local/python3/bin/python'` и автокомплит заработает

Так же для авто-форматирования необходимо установить и прописать гем erb-formatter, без него форматирование erb файлов не работает.

```
gem install erb-formatterОбъяснить с
```

так же мне пришлось сделать символьную ссылку в shims для rbenv чтоб оно все заработало.

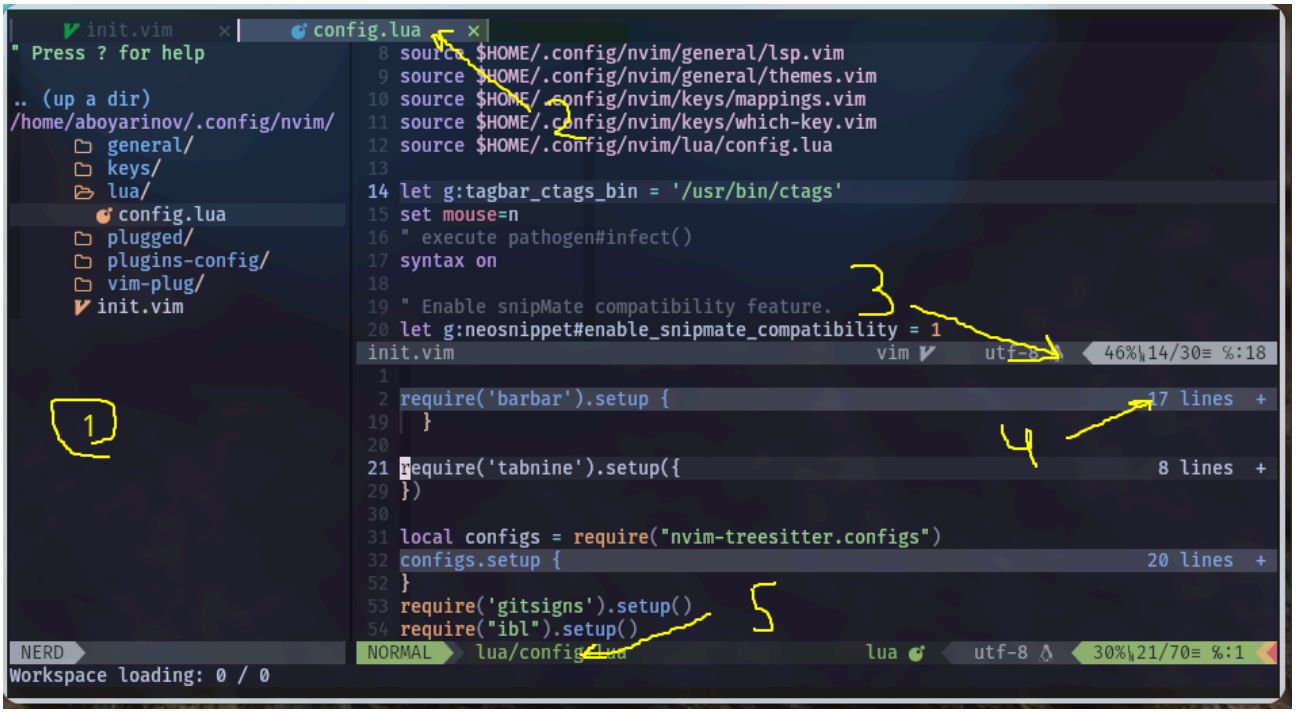
```
cd ~/.rbenv/shims && ln -s ~/.rbenv/verisons/3.3.0/bin/erb-formatter ./erb-formatterОбъяснить с
```

версию руби вместо 3.3.0 укажите ту которую используете в проекте. При сохранении файла, будет происходить авто-форматирование файла.

6. Обзор интерфейса

При запуске уже установленного и настроенного редактора мы видим

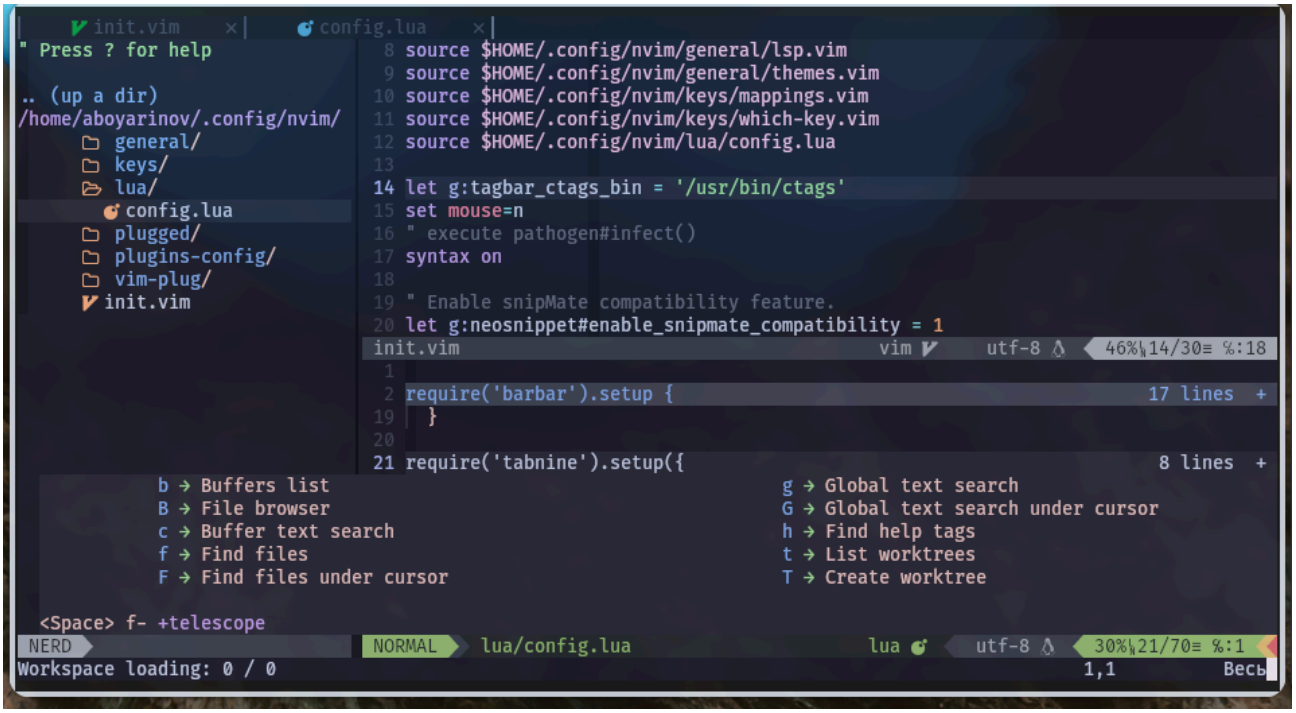
При нажатии на "Leader" - в нашей настройке это "пробел", видим список клавиш которые нам доступны для управления



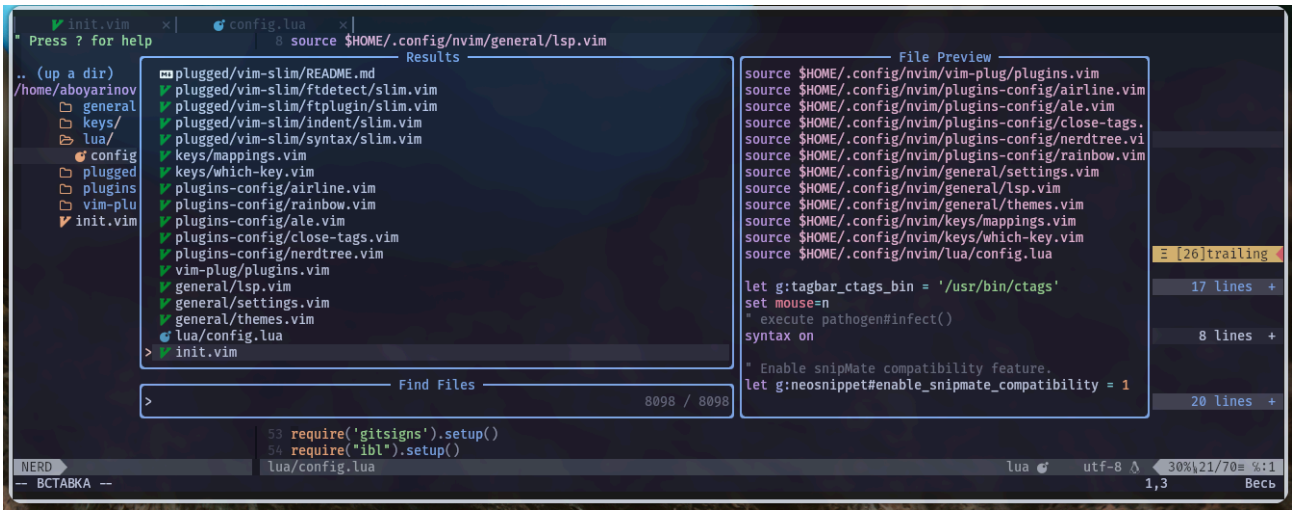
На общем скриншоте под номерами числятся:

1. NerdTree - файл менеджер для NeoVim (создание, удаление, копирование файлов и навигация по проекту)
2. Вкладки открытых файлов в редакторе
3. Серым цветом неактивный но открытый файл и его общая информация. Вся информация о файлах внизу открытого "окна"
4. Показывает что свернуто 17 строк внутри данной полоски, управление `z + 0` открыть все что вложено, `z + c` свернуть что можно от этой строки и глубже по иерархии
5. Статусная строка открытого и активного-редактируемого файла. информация о наименовании, в каком режиме редактор, какой язык и прочая нужная информация

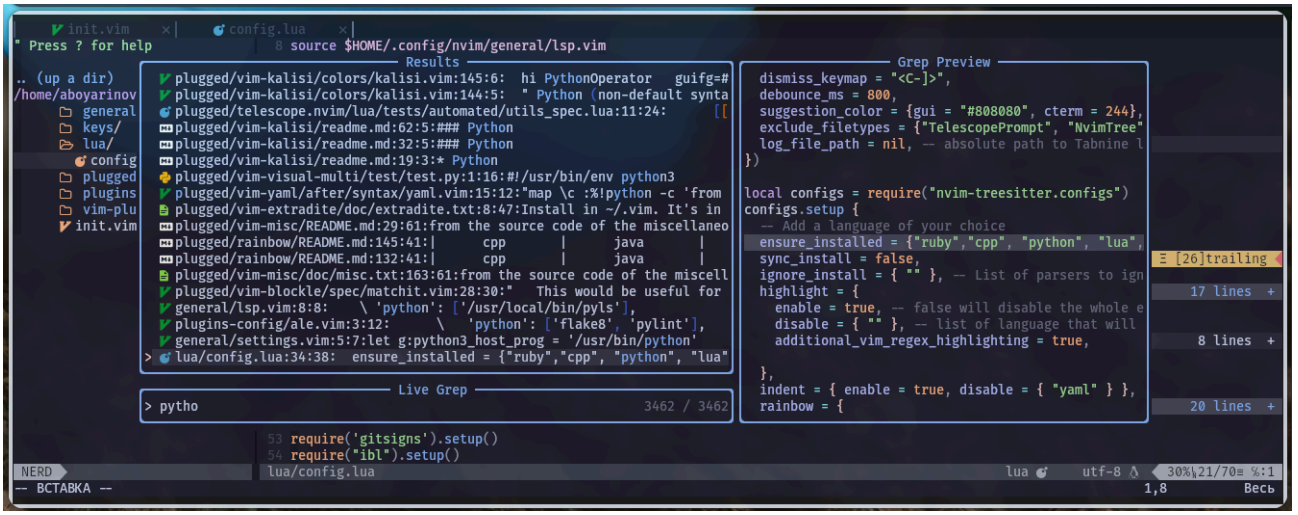
Большая часть всех интересных для разработчика инструментов спрятаны в "Telescope" при нажатии на `<Leader> + f` попадаем в меню:



далее нажимая на соответствующие символы слева от стрелок, вызываем определенные команды. Так для поиска по имени файлов в открытом проекте мы нажимаем `<leader> + f + f` и видим:

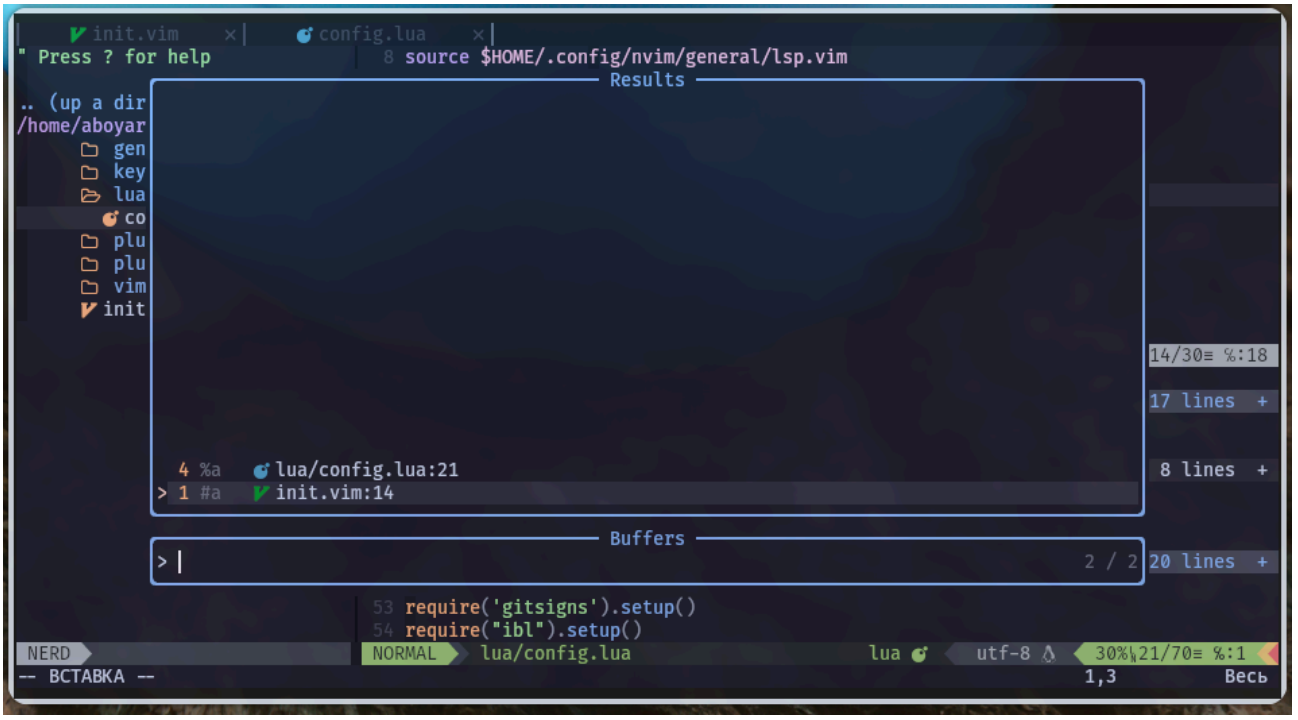


Поиск по содержимому файлов вызывается `<leader> + f + g`



Поиск по содержимому файлов

Поиск по открытым буферам (те файлы что редактировали ранее, но они скрылись из виду) `<leader> + f + b`



Поиск по открытым буферам

Остальное можно посмотреть уже и понять по смыслу наименований

7. Горячие клавиши для использования данной конфигурации:

Часть горячих клавиш не внесены в настройки плагина WhichKey, опишу их здесь ниже:

```
`c + s`      - изменить "окружающие" парные символы типа ковычек
`CTRL + n`   - выбор для мультиредактирования "переменных"
`g + c` -  комментирует/раскомментирует выделенные строки
`/` - поиск в открытом буфере
Объяснить c
```

8. Плагины используемые в данной настройке и описания к ним

- **Shougo/deoplete.nvim** - асинхронный фреймворк для авто-подстановки в NeoVim (выпадающее меню с предложениями)
- **tbodt/deoplete-tabnine** - подключение tabnine для deoplete, позволяет использовать AI при работе
- **Keithbsmiley/rspec.vim** - улучшенная подсветка синтаксиса для rspec
- **Shougo/neosnippet.vim** - добавление поддержки сниплетов для Vim
- **SirVer/ultisnips** - еще один вариант поддержки сниплетов для Vim
- **lukas-reineke/indent-blankline.nvim** - плагин для отображения вложенности, настраивается
- **lewis6991/gitsigns.nvim** - отображение данных из Git коммитов, удаленное, измененное наименования коммитов и т.п.
- **vim-scripts/VimCompletesMe** - очень легкий вариант авто-подстановки для редактора
- **alvan/vim-closetag** - автоматическое закрытие HTML тэгов
- **ap/vim-css-color** - отображение цветов в самом коде
- **bfredl/nvim-miniyank** - вспомогательный плагин для копирования и вставки текста
- **cespare/vim-toml** - поддержка синтаксиса toml
- **ctrlpvim/ctrlp.vim** - поисковик для файлов, буферов, тэгов и т.п. для Vim
- **dense-analysis/ale** - асинхронный линтер для neovim. позволяет использовать Go to definition. Используется с Solargraph для Rails проектов. является LSP клиентом, можно подключать необходимый функционал
- **dzeban/vim-log-syntax** - подсветка лог файлов
- **ekalinin/dockerfile.vim** - поддержка синтаксиса для docker файлов
- **etordera/deoplete-rails** - авто-подстановка для Rails проектов

- **morhetz/gruvbox** - цветовая схема Gruvbox
- **freeo/vim-kalisi** - цветовая схема kalisi
- **lifepillar/vim-gruvbox8** - цветовая схема Gruvbox8
- **catppuccin/nvim** - цветовая схема catppuccin
- **sainnhe/sonokai** - цветовая схема sonokai
- **hashivim/vim-terraform** - поддержка синтаксиса для файлов terraform
- **honza/vim-snippets** - поддержка сниплетов для vim
- **jiangmiao/auto-pairs** - автоматическая подставка парных символов "", '', (), {} и т.п.
- **juliosueiras/vim-terraform-completion** - авто-подстановка для файлов terraform
- **kchmck/vim-coffee-script** - поддержка синтаксиса coffee-script
- **iamcco/markdown-preview.nvim** - позволяет видеть отображение в браузере файла Markdown, который редактируете в данный момент
- **mfussenegger/nvim-lint** - линтер для множества языков, позволяет с комфортом работать в коде
- **mustache/vim-mustache-handlebars** - навигация между открывающими и закрывающими тегами
- **mxw/vim-jsx** - поддержка синтаксиса для jsx файлов
- **nelstrom/vim-textobj-rubyblock** - для создания "шаблонов" руби блоков. можно делать разные заготовки
- **othree/eregex.vim** - Обозначение regexp в стиле Perl/Ruby для Vim
- **othree/html5.vim** - поддержка синтаксиса для html5
- **othree/javascript-libraries-syntax.vim** - поддержка синтаксиса для javascript
- **othree/jspc.vim** - авто-подстановка для параметров Javascript
- **pseewald/vim-anyfold** - позволяет экономить место на экране, сворачивая вложенности. по умолчанию включено при открытии файла
- **rhysd/vim-grammarous** - проверка грамматики
- **rking/ag.vim** - интеграция Argentum Search(AG) в NeoVim

- **scrooloose/nerdtree** - навигация по проекту, проводник файлов
- **Xuyuanp/nerdtree-git-plugin** - отображение статуса git для файлов проекта
- **slim-template/vim-slim** - поддержка slim для NeoVim
- **stephpy/vim-yaml** - поддержка yaml файлов
- **antoinemadec/FixCursorHold.nvim** - для лучшего отображения курсора
- **nvim-neotest/neotest** - позволяет из редактора запускать тесты и работать с ними
- **olimorris/neotest-rspec** - поддержка rspec для neotest
- **tpope/vim-commentary** - комментирование и де комментирование строк (выделенного текста)
- **nvim-treesitter/nvim-treesitter** - очень мощный фреймворк поддерживающий большое количество языков программирования, синтаксис, линтеры и т.п.
- **tpope/vim-endwise** - автоматическая подстановка `end` после написания `def` `if` и тп. для Ruby
- **tpope/vim-fugitive** - поддержка Git для редактора
- **tpope/vim-rails** - улучшенная поддержка Rails проектов для NeoVim, навигации по ним и не только.
- **tpope/vim-surround** - для изменения парных символов, к примеру можно поменять с `"текст"` на `'текст'` очень просто. `c+s+"+'` находясь курсором на тексте внутри кавычек
- **kdheepak/lazygit.nvim** - очень удобное приложение для работы с Git репозиторием. Большой функционал. данный плагин позволяет запускать его не выходя из NeoVim
- **vim-airline/vim-airline** - статусная строка внизу каждого буфера
- **vim-airline/vim-airline-themes** - темы для статусной строки
- **vim-ruby/vim-ruby** - поддержка синтаксиса для Ruby
- **vim-syntastic/syntastic** - проверка синтаксиса для Vim более не поддерживается
- **xolox/vim-misc** - набор vim скриптов используемые множеством плагинов
- **xolox/vim-notes** - создание заметок непосредственно в NeoVim

- **declancm/cinnamon.nvim** - плавная прокрутка в редакторе
- **romgrk/barbar.nvim** - полоса сверху позволяет мышью управлять открытыми буферами, открывать закрывать менять положение
- **nvim-lua/plenary.nvim** - библиотека для Lua
- **nvim-tree/nvim-web-devicons** - поддержка devicons в NeoVim для NerdTree
- **ryanoasis/vim-devicons** - поддержка devicons в NeoVim
- **nvim-telescope/telescope.nvim** - удобный интерфейс для поиска всего и везде
- **nvim-telescope/telescope-live-grep-args.nvim** - поддержка Argentum Search для Telescope
- **liuchengxu/vim-which-key** - удобные подсказки по горячим клавишам, вызывается нажатием в данном случае это "Пробел"
- **luochen1990/rainbow** - разноцветные скобки, удобно при большой вложенности
- **codota/tabnine-nvim** - поддержка tabnine AI, возможности зависят от вашей подписки
- **mg979/vim-visual-multi** - превращает NeoVim в мульти-курсорный редактор
- **ThePrimeagen/git-worktree.nvim** - поддержка Git Worktree для NeoVim